



操作系统实验

题 目	:	实验5
专业名称	:	计算机科学与技术
授课教师	:	陈鹏飞
学生姓名	:	陈安康
学 号	:	22320021
实验地点	:	实验中心D402
日 期	:	2024/5/5

任务一：printf的实现

1.对于可变参机制的分析

对可变参数函数机制进行学习，该机制主要是依靠三个宏(还有一个指针定义，这里没算上)依靠指针来完成对栈上未知数目的参数的读取和传递。这三个宏分别的作用是初始化指针，读取参数和指针的移动，销毁指针。初始化指针依靠已知参数的地址来找到未知参数的初始地址，利用的是参数在栈上存放的规律以及地址运算，示例代码巧妙的地方是**通过与运算获取向下取整的四的倍数来代替除法与乘法运算从而获得不同类型与int类型对齐后的空间大小（因为栈要求内存对齐）**，使代码简洁高效；读取参数和指针移动就是简单的地址运算，这里示例代码巧妙的一点是**通过先递增指针为给定的类型（这里是内存对齐后的了）的长度使之指向下一个参数后，然后做减法得到返回的参数的地址再取地址内容**，这样就完成了递增指针指向下一个参数并且返回当前参数两个功能；销毁指针则是直接让指针指向地址为0，防止以后误用。

在完成上述三个宏的实现之后，结合之前实验中完成过的stdio类，就可以实现printf函数了。

2.printf函数的解释

在给定的printf函数中，对于输入的整型数据转化成不同进制的字符串类型，是用从栈上获取的参数除以进制，然后取得的余数依据进制转换成对应的字符，接着再用获得的商去进行上一步除进制的操作，以此往复直到商是0，接着将所得的字符串翻转即可得到对应进制下的字符串。

为什么给定的代码每次都需要调用printf_add_to_buffer将输出的字符放入缓冲区呢？**这是因为首先输出功能的实现依赖于缓冲区，真正的实现输出功能的函数接受的参数是一个数组的首地址，将字符放入缓冲区才能在之后调用stdio下的print方法（参数为数组地址的那一个，里面再进行'\n'的判断，然后再调用其重载函数print一个一个输出）进行字符的打印，不及时放入缓冲区的话得到的字符就丢失了，也就没有办法打印。其次缓冲区的大小是有限的，如果中途满了需要及时将字符打印出去清空，这就要求不能只是将字符放进buffer就行了，还要有判满以及后续的调用打印的相关操作，所以通过一个函数将以上要求实现的功能打包就十分方便。**

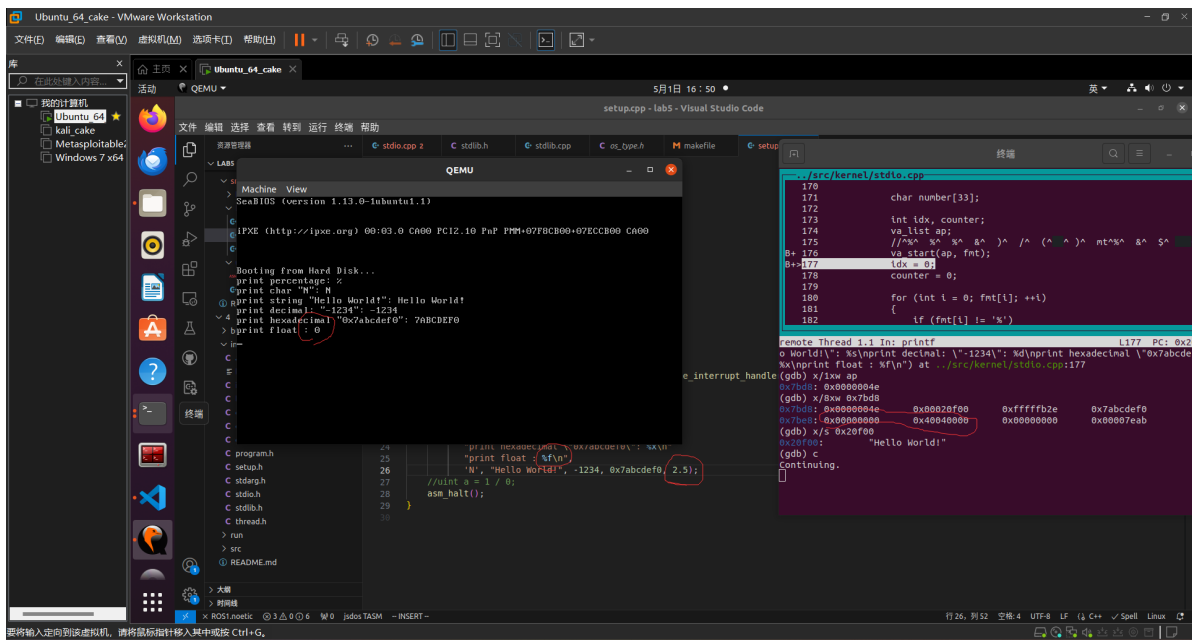
3.实现浮点数指定精度输出

学习过给定的printf函数之后，计划在给定代码的基础上进行扩展，增加格式化输出浮点数的功能，实现思路如下：

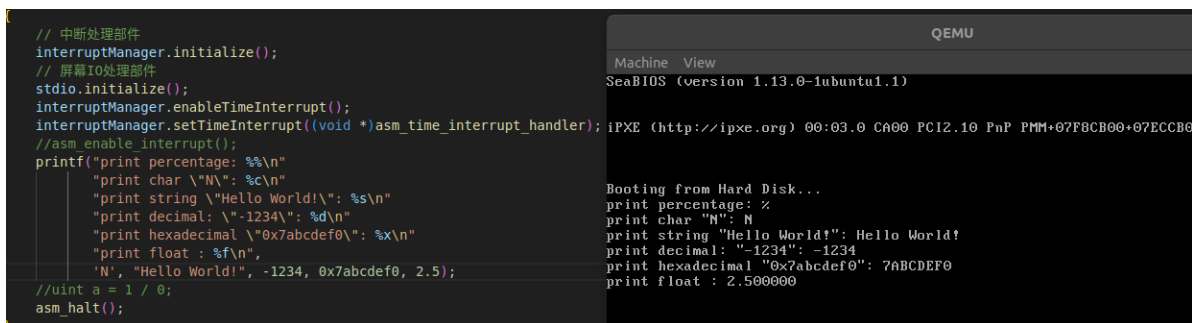
首先在前字符为'%'的基础上，判定当前字符为'f'(或者'F')，则通过va_arg以浮点数类型来读取下一个栈上的参数，通过函数ftos来将浮点数转换成字符串放在number数组空间中缓存（这里复用了一下存放整数时用的字符数组），之后将number字符放在buffer缓存区中打印出来。

ftos函数的实现思路：将浮点数分成整数与小数部分，分开进行转化，整数部分采用除10来取得每一位数字并转换成字符存放（这里只支持十进制表示），最后翻转；小数部分采用乘10取每位数字转换成字符存放。

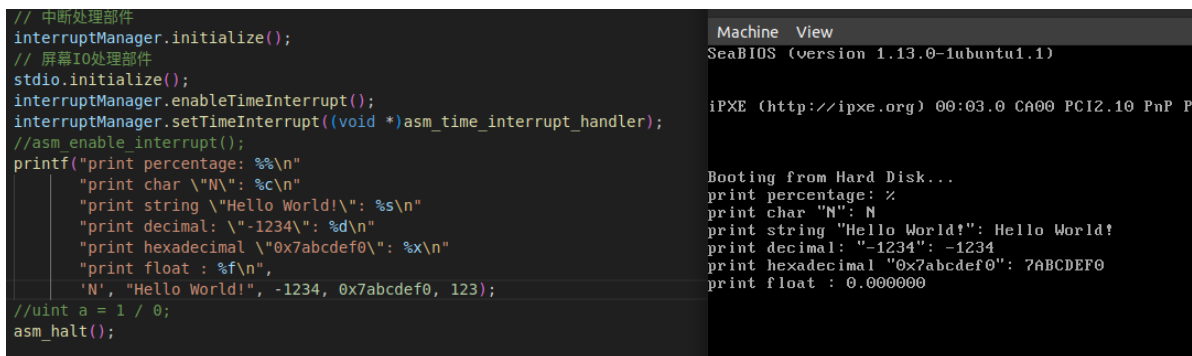
在实现过程中，遇到了一点小问题，在刚开始由于惯性思维，我将浮点数类型按照float类型进行操作，从栈上取数据也是float类型，结果出现如下情况：



可以看到预计输出为2.5,实际输出为0, 在旁边的gdb调试中列出来的是栈上的参数, 可以看到依次是'N'(0x4e是'N'的ASCII码), "Hello world"的首地址, -1234的16进制表示以及16进制输出的整数、按照预期, 下一个应该是按IEEE754标准存放的2.5 (即0x40200000), 但存放的却是0, 所以导致后续输出错误。经过我的仔细检查, 发现接下来的8个字节存放的是64位的IEEE754标准的double类型的数据即0x4004000000000000。原来在存放浮点数时, 默认是按照double类型存放的, 所以应该在读取时读取8个字节而不是float类型的4个字节, 在将浮点数读取类型和接下来的操作类型全部改为double类型后, 可以正常输出了:



并且注意到, 若是以浮点类型输出整型数, 会出错, 这是很自然的, 因为浮点类型是按照IEEE754标准形式存放的, 在放在栈上时, 如果数据是整型, 则会默认按补码形式存放, 在读取时按照浮点类型读取肯定会产生错误输出错误数据, 解决方法就是在后面加上小数部分, 如123想要按照浮点数形式输出, 要写成123.0。



```
// 中断处理部件
interruptManager.initialize();
// 屏幕IO处理部件
stdio.initialize();
interruptManager.enableTimeInterrupt();
interruptManager.setTimeInterrupt((void *)asm_time_interrupt_handler);
//asm enable interrupt();
printf("print percentage: %s\n"
       "print char \"M\": %c\n"
       "print string \"Hello World!\": %s\n"
       "print decimal: \"-1234\": %d\n"
       "print hexadecimal \"0x7abcdef0\": %x\n"
       "print float : %f\n",
       "N", "Hello World!", -1234, 0x7abcdef0, 123.0);

//uint a = 1 / 0;
asm_halt();
```

QEMU
Machine View
SeaBIOS (version 1.13.0-tubuntu1.1)
iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB
Booting from Hard Disk...
print percentage: %
print char "M": M
print string "Hello World!": Hello World!
print decimal: "-1234": -1234
print hexadecimal "0x7abcdef0": 7ABCDEF0
print float : 123.000000

在初次将操作的浮点类型从float修改为double类型时，进行编译出现如下报错：

```
user@user-virtual-machine: ~/lab5/3/build
ld -o kernel.o -melf_i386 -N entry.obj setup.o interrupt.o stdio.o stdlib.o asm_
utils.o -e enter_kernel -Ttext 0x00020000
ld: stdio.o: in function `printf(char const*, ...)':
/home/user/lab5/3/build/./src/kernel/stdio.cpp:217: undefined reference to `fto
s(char*, float)'
make: *** [makefile:46: kernel.o] 错误 1
user@user-virtual-machine:~/lab5/3/build$ make && make run
ld -o kernel.o -melf_i386 -N entry.obj setup.o interrupt.o stdio.o stdlib.o asm_
utils.o -e enter_kernel -Ttext 0x00020000
ld: stdio.o: in function `printf(char const*, ...)':
/home/user/lab5/3/build/./src/kernel/stdio.cpp:217: undefined reference to `fto
s(char*, float)'
make: *** [makefile:46: kernel.o] 错误 1
user@user-virtual-machine:~/lab5/3/build$ make clean
rm -f *.o* *.bin
user@user-virtual-machine:~/lab5/3/build$ make && make run
nasm -o mbr.bin -f bin -I../include/ ../src/boot/mbr.asm
nasm -o bootloader.bin -f bin -I../include/ ../src/boot/bootloader.asm
nasm -o entry.obj -f elf32 ../src/boot/entry.asm
g++ -g -Wall -march=i386 -m32 -nostdlib -fno-builtin -ffreestanding -fno-pic -I.
../include -c ../src/kernel/setup.cpp ../src/kernel/interrupt.cpp ../src/kernel/s
tdio.cpp ../src/utils/stdlib.cpp
nasm -o asm_utils.o -f elf32 ../src/utils/asm_utils.asm
ld -o kernel.o -melf_i386 -N entry.obj setup.o interrupt.o stdio.o stdlib.o asm_
utils.o -e enter_kernel -Ttext 0x00020000
objcopy -O binary kernel.o kernel.bin
dd if=mbr.bin of=../run/hd.img bs=512 count=1 seek=0 conv=notrunc
记录了1+0 的读入
记录了1+0 的写出
512字节已复制, 0.000191683 s, 2.7 MB/s
dd if=bootloader.bin of=../run/hd.img bs=512 count=5 seek=1 conv=notrunc
记录了0+1 的读入
```

将编译中间文件全部删除后重新编译就正常了，当时没有过多考虑出错原因，之后想要复现错误也没有复现出来，看报错和正常修改时的输出推断，应该还是没有及时更新cpp的目标文件（看下面的这一张图，可以看到修改后会先重新生成源文件所对应的目标文件然后再链接，而上面的出错图没有生成直接进行了链接），导致链接时依旧是老的float参数的函数文件导致没有找到函数声明导致的错误。

接下来要进行浮点数输出的精度的控制，可以用"`""%.xf"`来输出小数点后面的x位，只要稍微修改一下ftos函数，传入输出精度，默认精度为小数点后6位，再增加一个情况，即判断'.'，完成效果如下：

	QEMU
<pre>extern "C" void setup_kernel() { // 中断处理部件 interruptManager.initialize(); // 屏幕IO处理部件 stdio.initialize(); interruptManager.enableTimeInterrupt(); interruptManager.setTimeInterrupt((void *)asm_time_interrupt_handler); //asm enable interrupt(); printf("print percentage: %c\n" "print char \"N\": %c\n" "print string \"Hello World!\": %s\n" "print decimal: \"-1234\": %d\n" "print hexadecimal \"0x7abcdef0\": %x\n" "print float : %.3f\n", 'N', "Hello World!", -1234, 0x7abcdef0, 2.5); //uint a = 1 / 0; asm_halt(); }</pre>	<pre>Machine View SeaBIOS (version 1.13.0-1ubuntu1.1) iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP P Booting from Hard Disk... print percentage: % print char "N": N print string "Hello World!": Hello World! print decimal: "-1234": -1234 print hexadecimal "0x7abcdef0": 7ABCDEF0 print float : 2.500</pre>

4.结果展示

最后的代码：

```
void ftoa(char *numStr, double num, uint32 k) {
    uint32 length = 0, num_int = (int)num;
    // unsigned long long ieee754 = (*(unsigned long long*)&num) & (0x000fffffffffffff);
    // double num_float = *(double*)&ieee754;
    double num_float = num - (int)num;
    while(num_int) {
        uint32 tmp = num_int % 10;
        num_int /= 10;
        numStr[length] = tmp + '0';
        ++length;
    }
    if(!length) {
        numStr[length] = '0';
        ++length;
    }
    // 将字符串倒转，使得numStr[0]保存的是num的高位数字
    for(int i = 0, j = length - 1; i < j; ++i, --j) {
        swap(numStr[i], numStr[j]);
    }
    //小数部分不为零，输出小数部分，只输出6位，这部分不用反转
    // if(num_float) {
    // if(num_int == num) {
        numStr[length] = '.';
        ++length;
        num_float *= 10;
        // uint32 k = 3;
        while(k--) {
            numStr[length] = (int)num_float + '0';
            ++length;
            num_float = num_float - (int)num_float;
            num_float *= 10;
        }
    }
```

```
    }
    // }
    numStr[length] = '\0';
}
```

任务二：线程的实现以及问题探究（涉及到线程调度）

首先要实现PCB，教程中的PCB的数目是固定的，为每个PCB分配固定大小的空间，栈自带在PCB上，栈底默认为PCB的结尾，从高地址向低地址扩展，esp保存栈顶位置，在调度时将esp寄存器的值存储到stack指针中，以此在之后的调度中将esp恢复。除此之外，在pcb最开始的位置存放pid、名称、优先级、时间片、用来找寻PCB在队列中的位置的指针结构体等信息。

在这里如何通过链表指针找到PCB呢？这里利用了链表在PCB的相对位置是固定的这一规定，只要知道链表指针的地址，将该地址减去链表指针在PCB中的偏移量就能得到PCB的首地址，从而找到对应的PCB。

对宏传入参数的探究

这里我最初比较疑惑的一点是传入的第二个参数是哪里来的，因为这个tagInGeneralList很突兀，按理来说它应该依附于某一个特定的PCB才对，但这里直接将这个结构体成员传入。因此我自己实验了一下，自己用同样的方法找地址发现是正确的，的确可以不报错找到结构体的首地址：

```

9 using namespace std;
10 struct a {
11     int c;
12     int b;
13 };
14 #define c2a(c,c_) ( (a*)( (long long)&c) - (long long)&(((a*)0) -> c_) )
15 int main() {
16     // int n;
17     // double a;
18     // const char* const fmt = "cjhkh";
19     // int* b = &n;
20     // const char* p = "hello";
21     // cout << hex << (intptr_t)fmt << endl;
22     // cout << dec << sizeof(p) << endl;
23     // cout << sizeof(fmt) << endl;
24     // cout << _SIZE(fmt) << endl;
25     // cout << sizeof(b) << endl;
26     // int ptr = (intptr_t)fmt;
27     // printf("%s\n",ptr);
28     // printf("%s\n",fmt);
29     a a1;
30     a1.c = 2;
31     a1.b = 3;
32     cout << &a1 << endl;
33     long long q = (long long)c2a(a1.c,c);
34     cout << hex << q << endl;
35     return 0;
36 }

```

问题 输出 调试控制台 终端 端口

PS C:\Users\86135\Desktop\c++> & 'c:\Users\86135\.vscode\extensions\ms-vscode.cpptools-1.20.4-win32-x64\debugAdapters\bin\Window SDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-1e4zrhwx.w5p' '--stdout=Microsoft-MIEngine-Out-4jrvdzzv.cjh' '--stderr=Microsoft-MIEngine-Error-qmc32oj3.3je' '--pid=Microsoft-MIEngine-Pid-2wmibtkq.c5e' '--dbgExe=C:\Program Files\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.29.30133\bin\amd64\Debug\cl.exe' '--interpreter=mi' 0x61fe10 61fe10

之后查阅了相关资料，发现是这里的ListItem2PCB不是函数而是宏的原因，宏只是文本替换，没有所谓的类型检查，从而可以运行，如果这里变成函数那么实现上就会非常复杂。

其实这里PCB是结构体而不是类也隐形帮了忙，因为结构体的默认成员都是公有的，若是定义成类且为私有成员的话，不能直接访问，也会报错，如下图：

```
10 struct a {
11 private:
12     int b;
13     int c;
14 public:
15     int& getc() {
16         return c;
17     }
18 };
19 #define c2a(c,c_) ( (a*)( (long long)(&c) - (long long)&((a*)0) -> c_ ) )
20 int main() {
21     // int n;
22     // double a;
23     // const char* const fmt = "cjhhhh";
24     // int* b = &n;
25     // const char* p = "hello";
26     // cout << hex << (intptr_t)fmt << endl;
27     // cout <
28     a a1;
29     // a1.c = 2;
30     //a1.b = 3;
31     int& p = a1.getc();
32     cout << &a1 << endl;
33     // cout << &p << endl;
34     long long q = (long long)c2a(p,c);
35     // cout << hex << q << endl;
36     return 0;
37 }
```

问题 2 输出 调试控制台 终端 端口

leetcode.cpp 2

- 成员 "a::c" (已声明 所在行数:13) 不可访问 C/C++(265) [行 34, 列 34]
- 'int a::c' is private within this context gcc [行 40, 列 40]

报错成员不可访问，这时候若想要实现的话，需要将宏里面的直接调用改成一个返回成员引用的函数，这样宏的实现应该如下：

```
10 struct a {
11 private:
12     int b;
13     int c;
14 public:
15     int& getc() {
16         return c;
17     }
18 };
19 #define c2a(c,func) ( (a*)( (long long)(&c) - (long long)&((a*)0) -> func) )
20 int main() {
21     // int n;
22     // double a;
23     // const char* const fmt = "cjhhhh";
24     // int* b = &n;
25     // const char* p = "hello";
26     // cout << hex << (intptr_t)fmt << endl;
27     // cout <
28     a a1;
29     // a1.c = 2;
30     //a1.b = 3;
31     int& p = a1.getc();
32     cout << &a1 << endl;
33     // cout << &p << endl;
34     long long q = (long long)c2a(p,getc());
35     cout << hex << q << endl;
36     return 0;
37 }
```

问题 输出 调试控制台 终端 端口

```
PS C:\Users\86135\Desktop\c++> & 'c:\Users\86135\.vscode\extensions\ms-vscode.cpptools-1.20.4-win
sDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-ruigi5rr.5tt' '--stdout=Microsoft-MIEngine-Out-
ft-MIEngine-Error-2nnurn3n.qvn' '--pid=Microsoft-MIEngine-Pid-biicuzyd.qjq' '--dbgExe=C:\Program F
-seh-rt_v6-rev0\mingw64\bin\gdb.exe' '--interpreter=mi'
0x61fdf8
61fdf8
PS C:\Users\86135\Desktop\c++> |
```

这样返回的也是正确的结果。

在线程的实现这一节，在任务上要求自己实现PCB，但是经过学习发现基本的PCB的实现其实教程中基本功能已经很完善了（教程中提示添加优先级，但是其实教程中的PCB已经有了），经过学习教程之后再重新去从头到尾写一遍代码感觉效率太低，应该也不是实验原本的用意，所以本次是在原给定的PCB实现上进行功能的补充或者改进。

线程的创建

在此PCB上实现一个带参数的线程创建，传入两个整型参数，输出打印两个参数相加之后的结果，实现思路是创建一个参数类，成员包含构造函数和两个整型变量，在主函数中创建参数对象，创建线程时将参数传递给线程，从而完成参数的传递，代码与实现效果如下：


```
class param {
public:
    int a;
    int b;
    param() : a(0), b(0) {

    }
    param(int a, int b) : a(a),b(b) {

    }
};
```

```
void my_thread(void *arg) {
    int ans = (*(param*)arg).a + (*(param*)arg).b;
    printf("pid %d name \"%s\": the param passed to thread is %d %d \nthe sum is %d\n",programManager.running->pid, programManager.runn
    asm_halt();
}
```

```
param p(2,2);
// 创建第一个线程
int pid = programManager.executeThread(my_thread, &p, "my thread", 1);
if (pid == -1)
{
    printf("can not execute thread\n");
    asm_halt();
}
```

QEMU

Machine View

SeaBIOS (version 1.13.0-1ubuntu1.1)

iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB00 CA00

Booting from Hard Disk...

pid 0 name "my thread": the param passed to thread is 2 2

the sum is 4

—

这其中在代码实现上有一点需要注意，因为线程函数的参数全部为void* 类型，所以在参数时需要强制类型转换，以我的代码为例，如果写成(param)arg -> a是要报错的，原因是强制类型转换的优先级要低于->，同理写成(param)arg.a也会报错，所以要写成((param)arg) -> a或者是(*(param*)arg).a。

参数传递过程的探究

因为参数是放在PCB的栈上面的，运行时esp指向的是PCB的栈顶，我们在函数里面直接就取了参数了，并没有理会参数在这个过程中是怎么从PCB的栈上供函数去找寻到的，这里试着分析一下具体的调用逻辑：

首先要搞清楚这个栈存储的内容，从高地址到低地址依次是参数的首地址、program_exit()的首地址、函数的首地址、ebp、ebx、edi、esi寄存器的值，在gdb中追踪如下：

```
remote Thread 1.1 In: setup kernel
0x21e80 <PCB_SET>: 0x00022e64 0x7420796d 0x61657268 0x000000
64
0x21e90 <PCB_SET+16>: 0x00000000 0x00000000 0x00000002 0x000000
01
0x21ea0 <PCB_SET+32>: 0x00000000 0x0000000a 0x00000000 0x00031e
a8
0x21eb0 <PCB_SET+48>: 0x00000000 0x00031ea0 0x00000000 0x000000
--Type <RET> for more, q to quit, c to continue without paging--c
00
0x21ec0 <PCB_SET+64>: 0x00000000 0x00000000 0x00000000 0x00000000
(gdb) x/8xw 0x22e64
0x22e64 <PCB_SET+4068>: 0x00000000 0x00000000 0x00000000 0x00000000
0x22e74 <PCB_SET+4084>: 0x00020508 0x00020395 0x00007be0 0x00000000
(gdb) p &p
$3 = (param *) 0x7be0
(gdb) p my_thread
$4 = {void (void *)} 0x20508 <my_thread(void*)>
(gdb)
```

接下来执行到my_thread函数时（此时未进入），检查寄存器esp的值：

```
$3 = (param *) 0x7be0
(gdb) p my_thread
$4 = {void (void *)} 0x20508 <my_thread(void*)>
(gdb) b my_thread
Breakpoint 3 at 0x20508: file ../src/kernel/setup.cpp, line 47.
(gdb) i r esp
esp 0x7bd4 0x7bd4
(gdb) c
Continuing.

Breakpoint 3, my_thread (arg=0x7be0) at ../src/kernel/setup.cpp:47
(gdb) i r esp
esp 0x22e78 0x22e78 <PCB_SET+4088>
(gdb) x/2xw esp
No symbol "esp" in current context.
(gdb) x/2xw 0x22e78
0x22e78 <PCB_SET+4088>: 0x00020395 0x00007be0
(gdb)
```

可以发现esp指向的就是PCB的栈，而且已经恢复了四个寄存器和进入了线程函数，栈顶正好是应当返回的地址，接下来是参数，从lab4学习过的有关函数中栈的结构我们可以知道，pcb中的接下来的栈的结构就是线程函数的栈的结构，我们在存储时就是按照线程函数栈的结构去存储的，所以pcb的栈就是函数的栈，所以可以正确的找到参数。

线程函数返回地址的探究到asm_switch_thread函数的解析

可是这里我依旧有一些疑问，因为函数调用时会自动创建栈的，那那个栈呢？看汇编代码，发现函数会有创建栈的操作，继续往下走，进入函数，会发现esp和ebp都不是0了，而是另外的值：

```

Dump of assembler code for function my_thread(void*):
=> 0x00020508 <+0>:      endbr32
    0x0002050c <+4>:      push    ebp
    0x0002050d <+5>:      mov     ebp,esp
    0x0002050f <+7>:      push    ebx
    0x00020510 <+8>:      sub     esp,0x14
    0x00020513 <+11>:     mov     eax,DWORD PTR [ebp+0x8]
    0x00020516 <+14>:     mov     edx,DWORD PTR [eax]
    0x00020518 <+16>:     mov     eax,DWORD PTR [ebp+0x8]
    0x0002051b <+19>:     mov     eax,DWORD PTR [eax+0x4]
    0x0002051e <+22>:     add     eax,edx
    0x00020520 <+24>:     mov     DWORD PTR [ebp-0xc],eax
    0x00020523 <+27>:     mov     eax,DWORD PTR [ebp+0x8]
    0x00020526 <+30>:     mov     ecx,DWORD PTR [eax+0x4]
    0x00020529 <+33>:     mov     eax,DWORD PTR [ebp+0x8]
    0x0002052c <+36>:     mov     edx,DWORD PTR [eax]
    0x0002052e <+38>:     mov     eax,ds:0x31eb0
--Type <RET> for more, q to quit, c to continue without paging--

```

```

remote Thread 1.1 In: my_thread
    0x00020536 <+46>:     mov     eax,ds:0x31eb0
    0x0002053b <+51>:     mov     eax,DWORD PTR [eax+0x20]
    0x0002053e <+54>:     sub     esp,0x8
    0x00020541 <+57>:     push    DWORD PTR [ebp-0xc]
    0x00020544 <+60>:     push    ecx
    0x00020545 <+61>:     push    edx
    0x00020546 <+62>:     push    ebx
    0x00020547 <+63>:     push    eax
    0x00020548 <+64>:     push    0x216dc
    0x0002054d <+69>:     call    0x20e3c <printf(char const*, ...)>
    0x00020552 <+74>:     add     esp,0x20
    0x00020555 <+77>:     call    0x216ab <asm_halt>
    0x0002055a <+82>:     nop
    0x0002055b <+83>:     mov     ebx,DWORD PTR [ebp-0x4]
    0x0002055e <+86>:     leave
    0x0002055f <+87>:     ret
End of assembler dump.

```

```

remote Thread 1.1 In: my_thread
Breakpoint 4 at 0x20513: file ../src/kernel/setup.cpp, line 48.
(gdb) c
Continuing.

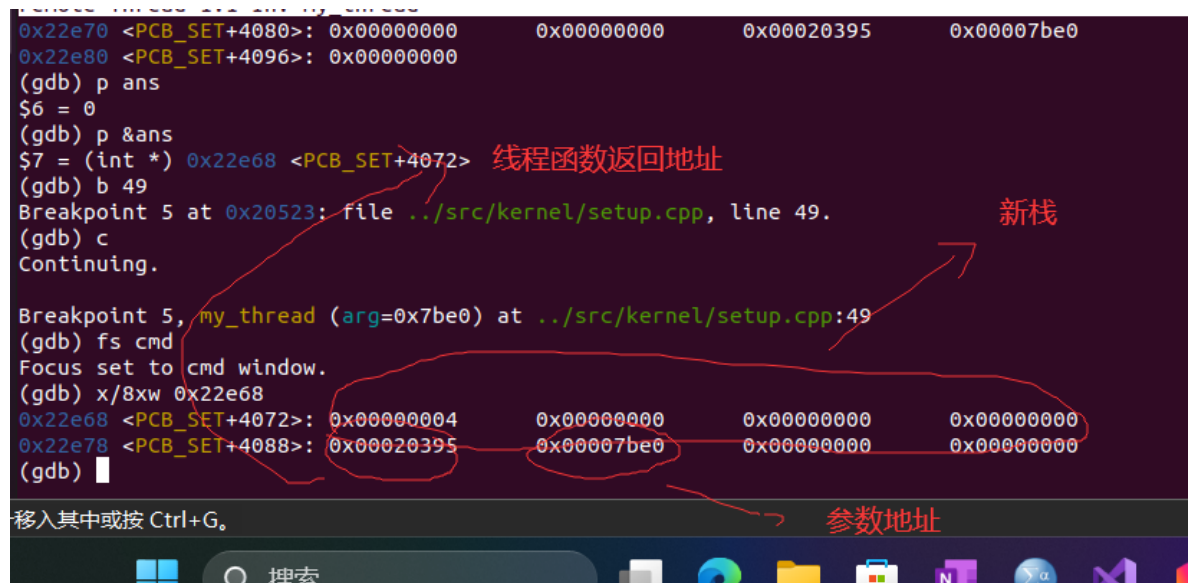
Breakpoint 4, my_thread (arg=0x7be0) at ../src/kernel/setup.cpp:48
(gdb) esp
Undefined command: "esp". Try "help".
(gdb) i r esp
esp                0x22e5c                0x22e5c <PCB_SET+4060>
(gdb) x/4xw 22e5c
Invalid number "22e5c".
(gdb) x/4xw 0x22e5c
0x22e5c <PCB_SET+4060>: 0x00000000      0x00000000      0x00000000      0x00000000
(gdb) p arg
$5 = (void *) 0x7be0
(gdb) i r ebp
ebp                0x22e74                0x22e74 <PCB_SET+4084>
(gdb)

```

可以看到函数以pcb的当前栈顶作为栈底，又创建了一个栈，栈顶到了0x22e5c了。最初我有些疑惑，后来经过反复思考和参照之前的lab4的教程，我明白了我之前的知识有一点混淆，正常的CPP调用函数是下面的过程：

push param ; 放参数到栈上，在这里初始化pcb时对栈的结构安排相当于自动完成这一条了

call function; 此时将返回地址放在栈上然后跳转到被调用函数的第一条指令，接下来在调用的函数里面创建新栈



The screenshot shows a GDB session with the following commands and output:

```
0x22e70 <PCB_SET+4080>: 0x00000000 0x00000000 0x00020395 0x00007be0
0x22e80 <PCB_SET+4096>: 0x00000000
(gdb) p ans
$6 = 0
(gdb) p &ans
$7 = (int *) 0x22e68 <PCB_SET+4072>
(gdb) b 49
Breakpoint 5 at 0x20523: file ../src/kernel/setup.cpp, line 49.
(gdb) c
Continuing.

Breakpoint 5, my_thread (arg=0x7be0) at ../src/kernel/setup.cpp:49
(gdb) fs cmd
Focus set to cmd window.
(gdb) x/8xw 0x22e68
0x22e68 <PCB_SET+4072>: 0x00000004 0x00000000 0x00000000 0x00000000
0x22e78 <PCB_SET+4088>: 0x00020395 0x00007be0 0x00000000 0x00000000
(gdb)
```

Annotations in the image:

- A red arrow points to the address `0x22e68 <PCB_SET+4072>` with the label "线程函数返回地址" (Thread function return address).
- A red arrow points to the address `0x00007be0` in the memory dump with the label "新栈" (New stack).
- A red arrow points to the address `0x00020395` in the memory dump with the label "参数地址" (Parameter address).

所以此时栈上的结构从高地址到低地址是：参数、program_exit()入口地址（这里当做是线程函数执行完后的返回地址）、新栈。这和我们的目前的结构一样（上图），所以是对的，理论与实际是对得上的。唯一疑惑的一点是返回地址，如果是call指令去调用的话，返回地址应该是调用函数的下一条语句，这里我们希望的返回地址是自己规定的，所以这里肯定不是用的call指令进入的函数，而且这里已经将返回地址放好了。教程上说是强制执行的，我不太理解这里，所以决定找一下进入线程函数之前的上一条指令在哪，我以为会有一个jmp指令去跳转，结果一条一条汇编执行下来，真的是直接跳转过去了，中间没有任何跳转指令。之后我明白了，其实，是asm_switch_thread里面将esp从一个栈换到了另一个栈的缘故，此时换到了当前进程上的栈，然后出栈恢复寄存器后，执行ret语句，这时栈顶是线程函数的首地址，ret指令把这个地址当做返回地址进行返回了，所以巧妙的就切换到了线程函数的首地址上。这样，既可以进入线程函数，又能保证线程函数的返回地址是我们预期的地址。

```
终端
+
./src/utils/asm_utils.asm
25      push ebx
26      push edi
27      push esi
28
29      mov eax, [esp + 5 * 4]
30      mov [eax], esp ; ^$^ ^ %^ %^ %^ &^ &^ )^ %^ CB^$^
31
32      mov eax, [esp + 6 * 4]
33      mov esp, [eax] ; ^&^ &^ %^ '^ $^ ur^&^ %^ &^ %
34
>35      pop esi
36      pop edi
37      pop ebx

remote Thread 1.1 In: asm_switch_thread L35 PC: 0x215bc
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
asm_switch_thread () at ./src/utils/asm_utils.asm:35
(gdb) i r esp
esp          0x22e64          0x22e64 <PCB_SET+4068>
(gdb) x/8xw 0x22e64
0x22e64 <PCB_SET+4068>: 0x00000000      0x00000000      0x00000000      0x000000
000
0x22e74 <PCB_SET+4084>: 0x00020508      0x00020395      0x00007be0      0x000000
000
(gdb) █

完成了两个PCB上的栈的切换

这个线程函数的入口地址，充当了
asm_switch_thread的返回地址
```

那之前的返回地址呢？因为中途有一个切换，使得最初调用asm_switch_thread函数的函数“戛然而止”，如果该线程函数执行完了，那么返回到的是我们规定的program_exit()上，这符合我们预期。如果还没有执行完就调用asm_switch_thread函数进行切换，当前栈上的内容从高往第依次是：参数、program_exit()入口地址、asm_switch_thread的返回地址（也就是接下来应该执行的线程函数的内容起始位置）、切换之前的寄存器的值。

所以当某个asm_switch_thread又切回这个pcb执行时，返回地址刚好又是接下来线程函数要执行的指令的地址！就这样，通过asm_switch_thread,巧妙完成了切换、保护现场两个功能。

至此，对于PCB的结构分析以及线程的创建已经分析完毕，接下来就是线程调度的分析和实现，因为前面已经初步分析过asm_switch_thread函数的作用，接下来这一方面不再赘述。

任务三：线程的调度

测试结果展示

这里写一个测试代码，包括三个线程，定义如下：

```

void my_thread_schedule0(void* arg) {
    // int k = 3;
    // while(k--) {
    //     printf("the running thread is 0\n");
    // }
    // return ;
    long long count = 0;
    while(true) {
        count ++;
        if(count == 100000000) {
            printf("the running thread is 0\n");
            count = 0;
        }
    }
};
}

```

```

//用于gdb调试的线程，有死循环
void thread1(void* arg) {
    int count = 0;
    while(true) {
        ++count;
        if(count == 100000000) {
            printf("this is thread for thread schedule debug,this is 1\n");
            count = 0;
        }
    }
}

//用于gdb调试的线程，没有死循环
void thread2(void* arg) {
    printf("this is thread for thread schedule debug,this is 2\n");
}

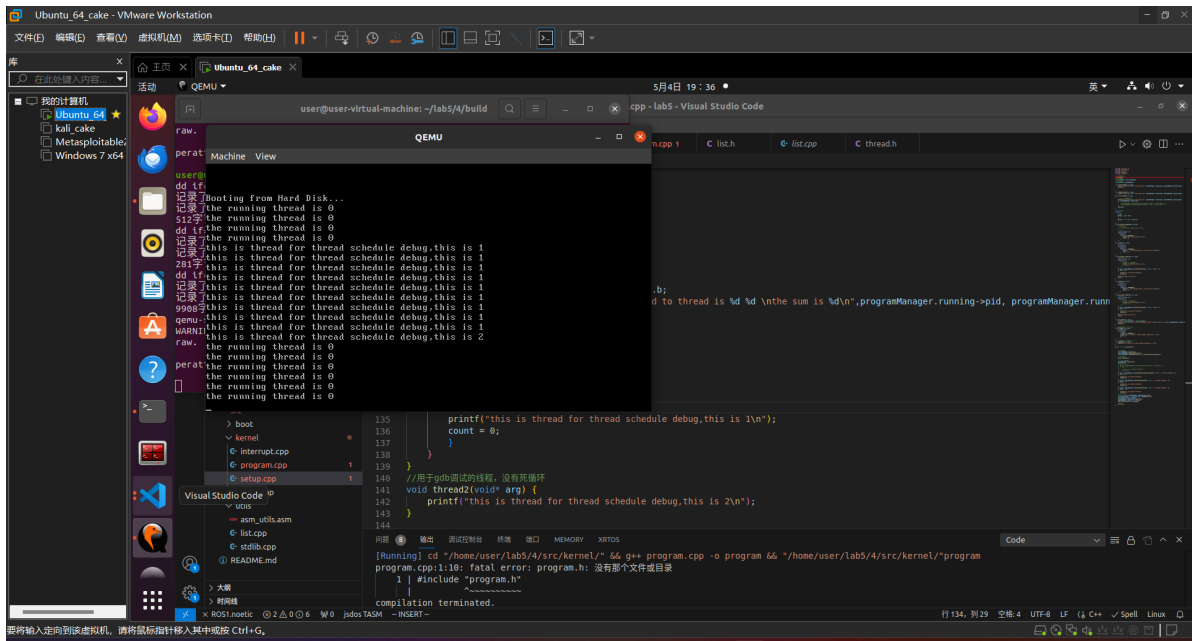
```

```

int pid = programManager.executeThread(my_thread_schedule0, nullptr, "my thread schedule0", 1);
if (pid == -1)
{
    printf("can not execute thread\n");
    asm_halt();
}
int pid1 = programManager.executeThread(thread1, nullptr, "my thread schedule1", 1);
if (pid1 == -1)
{
    printf("can not execute thread\n");
    asm_halt();
}
int pid2 = programManager.executeThread(thread2, nullptr, "my thread schedule2", 1);
if (pid2 == -1)
{
    printf("can not execute thread\n");
    asm_halt();
}
}

```

效果如下：



利用GDB追踪切换过程

进入gdb, 在asm_switch_thread、c_time_interrupt_handler、schedule函数打上断点, 执行, 第一个进入的是asm_switch_thread函数, 这里是将第一个进程放上cpu, 所以第一个参数是0, 第二个参数是第一个PCB的地址:


```
../src/utils/asm_utils.asm
19     ASM_IDTR dw 0
20         dd 0
21
22     ; void asm_switch_thread(PCB *cur, PCB *next);
23     asm_switch_thread:
B+ 24         push ebp
25         push ebx
26         push edi
27         push esi
28
29         mov eax, [esp + 5 * 4]
30         mov [eax], esp ; ^$^ ^ %^ %^ %^ &^ &^ )^ %^ CB^$^
31
32         mov eax, [esp + 6 * 4]
33         mov esp, [eax] ; ^&^ &^ &^ %^ ' ^ $^ ur^&^ %^ &^
34
35         pop esi
36         pop edi
37         pop ebx
38         pop ebp
39
40         sti
41         ret

remote Thread 1.1 In: asm_switch_thread
Breakpoint 3 at 0x20ee6: file ../src/kernel/interrupt.cpp, line 89.
(gdb) c
Continuing.

Breakpoint 1, asm_switch_thread () at ../src/utils/asm_utils.asm:24
(gdb) i r esp
esp                0x7bc0                0x7bc0
(gdb) i r ebp
ebp                0x7bfc                0x7bfc
(gdb) b 32
Breakpoint 4 at 0x21b56: file ../src/utils/asm_utils.asm, line 32.
(gdb) c
Continuing.

Breakpoint 4, asm_switch_thread () at ../src/utils/asm_utils.asm:32
(gdb) i r eax
eax                0x0                    0
(gdb) █
```

再往下执行，可以看到esp已经指向了第一个PCB的栈顶：


```
../src/utils/asm_utils.asm
19     ASM_IDTR dw 0
20         dd 0
21
22     ; void asm_switch_thread(PCB *cur, PCB *next);
23     asm_switch_thread:
B+ 24         push ebp
25         push ebx
26         push edi
27         push esi
28
29         mov eax, [esp + 5 * 4]
30         mov [eax], esp ; ^$^ ^ % ^ & ^ & ^ ) ^ % ^ CB^$^ / ^ $ ^ $ ^ ^ ^ & ^ % ^ & ^ ^ % ^
31
B+ 32         mov eax, [esp + 6 * 4]
33         mov esp, [eax] ; ^& ^ & ^ % ^ ' ^ $ ^ ur^& ^ % ^ & ^ % ^ ext^& ^
34
B+> 35         pop esi
36         pop edi
37         pop ebx
38         pop ebp
39
40         sti
41         ret

remote Thread 1.1 In: asm_switch_thread
ebp      0x7bfc      0x7bfc
(gdb) b 32
Breakpoint 4 at 0x21b56: file ../src/utils/asm_utils.asm, line 32.
(gdb) c
Continuing.

Breakpoint 4, asm_switch_thread () at ../src/utils/asm_utils.asm:32
(gdb) i r eax
eax      0x0      0
(gdb) b 35
Breakpoint 5 at 0x21b5c: file ../src/utils/asm_utils.asm, line 35.
(gdb) c
Continuing.

Breakpoint 5, asm_switch_thread () at ../src/utils/asm_utils.asm:35
(gdb) i r esp
esp      0x236a4      0x236a4 <PCB_SET+4068>
(gdb) █
```

接下来返回后，将进入第一个线程函数执行：

```
../src/utils/asm_utils.asm
30      mov [eax], esp ; ^$^ ^ %^ %^ %^ &^ &^ )^ %^ CB^$^ / ^ $^ $^
31
B+ 32      mov eax, [esp + 6 * 4]
33      mov esp, [eax] ; ^&^ &^ &^ %^ ' ^ $^ ur^&^ %^ &^ %^ ext^&^
34
35      pop esi
36      pop edi
37      pop ebx
38      pop ebp
39
>40      sti
41      ret
42      ; int asm_interrupt_status();
43      asm_interrupt_status:
44          xor eax, eax
45          pushfd
46          pop eax
47          and eax, 0x200
48          ret
49
50      ; void asm_disable_interrupt();
51      asm_disable_interrupt:
52          cli

remote Thread 1.1 In: asm_switch_thread
asm_switch_thread () at ../src/utils/asm_utils.asm:36
(gdb) i r esp
esp          0x236a8          0x236a8 <PCB_SET+4072>
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:37
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:38
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:40
(gdb) i r esp
esp          0x236b4          0x236b4 <PCB_SET+4084>
(gdb) x/4xw 0x236ba
0x236ba <PCB_SET+4090>: 0x00000002          0x46a40000          0x796d0002          0x72687420
(gdb) x/4xw 0x236b4
0x236b4 <PCB_SET+4084>: 0x000207e4          0x00020672          0x00000000          0x000246a4
(gdb) x/i 0x207e4
0x207e4 <my_thread_schedule0(void*)>:      endbr32
(gdb) █
```

接着执行进入时钟中断:

```
../src/kernel/interrupt.cpp
84     setInterruptDescriptor(IRQ0_8259A_MASTER, (uint32)handler, 0);
85 }
86
87 // ^$^ &^ %^ ' ^ %^ &^
88 extern "C" void c_time_interrupt_handler()
89 {
90     // ^% ^ % ^ ' ^ CFS ^ ' ^ & ^ % ^ & ^ ) ^ $ ^ & ^ & ^ % ^ ( ^ ^ / ^ & ^ ^ $ ^
91     PCB *cur = programManager.running;
92
93     if (cur->ticks)
94     {
95         --cur->ticks;
96         ++cur->ticksPassedBy;
97     }
98     else
99     {
100         programManager.schedule();
101     }
102
103     // // ^$ ^ ) ^ & ^ SA ^ & ^ % ^ % ^ ' ^ & ^ ' ^ & ^ ) ^ $ ^ & ^ / ^ ( ^ ^ ) ^ ' ^ % ^ ' ^
104     // PCB* cur = programManager.running;
105     // if (cur -> ticks) {
106     //     --cur -> ticks;
```

remote Thread 1.1 In: c_time_interrupt_handler
Continuing.

Breakpoint 3, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) c
Continuing.

Breakpoint 7, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:96
(gdb) c
Continuing.

Breakpoint 3, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) c
Continuing.

Breakpoint 7, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:96
(gdb) p cur->ticks
\$6 = 4
(gdb)

鼠标指针移入其中或按 Ctrl+G

时间片用完后进入调度函数:

```

1+1
终端
../src/kernel/interrupt.cpp
84         setInterruptDescriptor(IRQ0_8259A_MASTER, (uint32)handler, 0);
85     }
86
87     // ^$^ &^ %^ ' ^ %^ &^
88     extern "C" void c_time_interrupt_handler()
B+ 89     {
90         // ^% ^ % ^ ' ^ CFS ^ ^ & ^ & ^ % ^ & ^ ) ^ $ ^ & ^ & ^ % ^ ( ^ ^ / ^ &
91         PCB *cur = programManager.running;
92
93         if (cur->ticks)
94         {
95             --cur->ticks;
B+ 96             ++cur->ticksPassedBy;
97         }
98         else
99         {
B+>100             programManager.schedule();
101         }
102
103         // // ^$^ ) ^ & ^ SA ^ & ^ % ^ % ^ ' ^ & ^ ' ^ & ^ ) ^ $ ^ & ^ / ^ ( ^ ^ ) ^ '
104         // PCB* cur = programManager.running;
105         // if(cur -> ticks) {
106         //     --cur -> ticks;

```

remote Thread 1.1 In: c_time_interrupt_handler
Continuing.

Breakpoint 3, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) c
Continuing.

Breakpoint 7, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:96
(gdb) c
Continuing.

Breakpoint 3, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) c
Continuing.

Breakpoint 6, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:100
(gdb) p cur -> ticks
\$7 = 0
(gdb)

接下来进入调度函数，这里是新创建未执行过的线程函数线程1（next）与第一个线程（cur）之间进行切换：

```
终端 5月4日 20:49
终端
../src/kernel/program.cpp
90     if (running->status == ProgramStatus::RUNNING)
91     {
92         running->status = ProgramStatus::READY;
93         running->ticks = running->priority * 10;
94         readyPrograms.push_back(&(running->tagInGeneralList));
95     }
96     else if (running->status == ProgramStatus::DEAD)
97     {
98         releasePCB(running);
99     }
100
101     ListItem *item = readyPrograms.front();
102     PCB *next = ListItem2PCB(item, tagInGeneralList);
103     PCB *cur = running;
104     next->status = ProgramStatus::RUNNING;
105     running = next;
106     readyPrograms.pop_front();
107
B+>108     asm switch_thread(cur, next);
109
110     interruptManager.setInterruptStatus(status);
111 }
112

remote Thread 1.1 In: ProgramManager::schedule
$7 = 0
(gdb) c
Continuing.

Breakpoint 2, ProgramManager::schedule (this=0x326e0 <programManager>) at ../src/kernel/program.cpp:80
(gdb) fs src
Focus set to src window.
(gdb) b 108
Breakpoint 8 at 0x2036d: file ../src/kernel/program.cpp, line 108.
(gdb) c
Continuing.

Breakpoint 8, ProgramManager::schedule (this=0x326e0 <programManager>) at ../src/kernel/program.cpp:108
(gdb) p cur
$8 = (PCB *) 0x226c0 <PCB_SET>
(gdb) p next
$9 = (PCB *) 0x236c0 <PCB_SET+4096>
(gdb)
```

这个esp后面四个寄存器的值之后，就是下次调度到第一个进程后需要返回的地方，即schedule函数处，等再调度到该线程函数，则从该返回地址处继续往下执行。

```
utils.asm
Thunderbird 邮件/新闻 >al asm_switch_thread
15
16     extern c_time_interrupt_handler
17     ASM_UNHANDLED_INTERRUPT_INFO db 'Unhandled interrupt happened, halt...'
18                                   db 0
19     ASM_IDTR dw 0
20             dd 0
21
22     ; void asm_switch_thread(PCB *cur, PCB *next);
23     asm_switch_thread:
B+ 24         push ebp
25         push ebx
26         push edi
27         push esi
28
29         mov eax, [esp + 5 * 4]
30         mov [eax], esp ; ^$^ ^ %^ %^ &^ &^ )^ %^ CB^$^ / ^ $^ $^ ^
31
B+> 32         mov eax, [esp + 6 * 4]
33         mov esp, [eax] ; ^&^ &^ %^ ' ^ $^ ur^&^ %^ &^ %^ ext^&^
34
B+ 35         pop esi
36         pop edi

Remote Thread 1.1 In: asm_switch_thread
(gdb) c
Continuing.

Breakpoint 4, asm_switch_thread () at ../src/utils/asm_utils.asm:32
(gdb) i r esp
esp                0x235fc                0x235fc <PCB_SET+3900>
(gdb) x/4xw 0x235fc
0x235fc <PCB_SET+3900>: 0x00ab5d2a      0x00000000      0x00000000      0x00023638
(gdb) x/i 0xab5d2a
Invalid number "0xab5d2a".
(gdb) x/i 0xab5d2a
0xab5d2a: add BYTE PTR [eax],al
(gdb) x/8xw 0x235fc
0x235fc <PCB_SET+3900>: 0x00ab5d2a      0x00000000      0x00000000      0x00023638
0x2360c <PCB_SET+3916>: 0x0002037b      0x000226c0      0x000236c0      0x00000020
(gdb) x/i 0x2037b
0x2037b <ProgramManager::schedule()+307>: add esp,0x10
(gdb) █
```

然后进行栈的转换，转到线程1上，可以看到return后就要返回到线程1的线程函数进行执行，这样就完成了线程的切换。

```
../src/utils/asm_utils.asm
```

```

B+ 24      push ebp
    25      push ebx
    26      push edi
    27      push esi
    28
    29      mov eax, [esp + 5 * 4]
    30      mov [eax], esp ; ^$^ ^ % ^ % ^ & ^ & ^ ) ^ % ^ CB^$^ / ^ $^ $^ ^ ^ & ^ % ^ &
    31
B+ 32      mov eax, [esp + 6 * 4]
    33      mov esp, [eax] ; ^& ^ & ^ % ^ ' ^ $^ ur^& ^ % ^ & ^ % ^ ext^& ^
    34
B+>35      pop esi
    36      pop edi
    37      pop ebx
    38      pop ebp
    39
    40      sti
    41      ret
    42      ; int asm_interrupt_status();
    43      asm_interrupt_status:
    44      xor eax, eax
    45      pushfd
    46      pop eax

```

```
remote Thread 1.1 In: asm_switch_thread
```

```

0x235fc <PCB_SET+3900>: 0x00ab5d2a      0x00000000      0x00000000      0x00023638
0x2360c <PCB_SET+3916>: 0x0002037b      0x000226c0      0x000236c0      0x00000020

```

```
(gdb) x/i 0x2037b
```

```
0x2037b <ProgramManager::schedule()+307>: add esp,0x10
```

```
(gdb) c
```

```
Continuing.
```

```
Breakpoint 5, asm_switch_thread () at ../src/utils/asm_utils.asm:35
```

```
(gdb) i r esp
```

```
esp      0x246a4      0x246a4 <PCB_SET+8164>
```

```
(gdb) x/4xw 0x246a4
```

```
0x246a4 <PCB_SET+8164>: 0x00000000      0x00000000      0x00000000      0x00000000
```

```
(gdb) x/8xw 0x246a4
```

```
0x246a4 <PCB_SET+8164>: 0x00000000      0x00000000      0x00000000      0x00000000
```

```
0x246b4 <PCB_SET+8180>: 0x00020a36      0x00020672      0x00000000      0x000256a4
```

```
(gdb) x/i 0x20a36
```

```
0x20a36 <thread1(void*)>: endbr32
```

```
(gdb) █
```



```
../src/kernel/setup.cpp
120     return ;
121 }
122
123 void my_thread(void *arg) {
124     int ans = (*(param*)arg).a + (*(param*)arg).b;
125     printf("pid %d name \"%s\": the param passed to thread is %d\n",
126           asm_halt());
127 }
128
129 //'^^ $^ db^(^ (^ '^ '^ '^'^ /&^ &^ %^ '^ '^
>130 void thread1(void* arg) {
131     int count = 0;
132     while(true) {
133         ++count;
134         if(count == 10000000) {
135             printf("this is thread for thread schedule debug,this is %d\n", count);
136             count = 0;
137         }
138     }
139 }
140 //'^^ $^ db^(^ (^ '^ '^ '^'^ /&^ &^ &^ %^ '^ '^
141 void thread2(void* arg) {
142     printf("this is thread for thread schedule debug,this is 2\n");
}

remote Thread 1.1 In: thread1
0x226c0 <PCB_SET>: cld
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:36
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:37
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:38
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:40
(gdb) i r eip
eip          0x21b60          0x21b60 <asm_switch_thread+20>
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:41
(gdb) ni
thread1 (arg=0x0) at ../src/kernel/setup.cpp:130
(gdb) i r eip
eip          0x20a36          0x20a36 <thread1(void*)>
(gdb) █
```

可以看到转到了线程1的线程函数进行执行。

接下来一直执行，直到轮到第一个进程再被换上去时：

```

../src/kernel/program.cpp
99         }
100
101         ListItem *item = readyPrograms.front();
102         PCB *next = ListItem2PCB(item, tagInGeneralList);
103         PCB *cur = running;
104         next->status = ProgramStatus::RUNNING;
105         running = next;
106         readyPrograms.pop_front();
107
B+>108         asm_switch_thread(cur, next);
109
110         interruptManager.setInterruptStatus(status);
111     }
112
113     void ProgramManager::FCFS() {
114         bool status = interruptManager.getInterruptStatus();
115         interruptManager.disableInterrupt();
116         releasePCB(running);
117         if (readyPrograms.size() == 0)
118         {
119             interruptManager.setInterruptStatus(status);
120             //'^^ %^ ^ '^ %^ )^ %^ $^ $^ '^
121             while(readyPrograms.size() == 0);

```

remote Thread 1.1 In: ProgramManager::schedule

```

Breakpoint 3, ProgramManager::schedule (this=0x326e0 <programManager>)
    at ../src/kernel/program.cpp:108
(gdb) p cur
$1 = (PCB *) 0x246c0 <PCB_SET+8192>
(gdb) p next
$2 = (PCB *) 0x226c0 <PCB_SET>
(gdb)

```

Ubuntu_64_cake

5月4日 21:34

终端

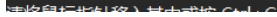
```
../src/kernel/program.cpp
93         running->ticks = running->priority * 10;
94         readyPrograms.push_back(&(running->tagInGeneralList));
95     }
96     else if (running->status == ProgramStatus::DEAD)
97     {
98         releasePCB(running);
99     }
100
101     ListItem *item = readyPrograms.front();
102     PCB *next = ListItem2PCB(item, tagInGeneralList);
103     PCB *cur = running;
104     next->status = ProgramStatus::RUNNING;
105     running = next;
106     readyPrograms.pop_front();
107
108     asm_switch_thread(cur, next);
109
110     interruptManager.setInterruptStatus(status);
111 }
112
113 void ProgramManager::FCFS() {
114     bool status = interruptManager.getInterruptStatus();
115     interruptManager.disableInterrupt();

```

remote Thread 1.1 In: ProgramManager::schedule

```
asm_switch_thread () at ../src/utils/asm_utils.asm:36
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:37
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:38
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:40
(gdb) i r eip
eip          0x21b60          0x21b60 <asm_switch_thread+20>
(gdb) ni
asm_switch_thread () at ../src/utils/asm_utils.asm:41
(gdb) i r eip
eip          0x21b61          0x21b61 <asm_switch_thread+21>
(gdb) ni
0x0002037b in ProgramManager::schedule (this=0x326e0 <programManager>) at ../src/kernel/program.cpp:108
(gdb) i r eip
eip          0x2037b          0x2037b <ProgramManager::schedule()+307>
(gdb)

```

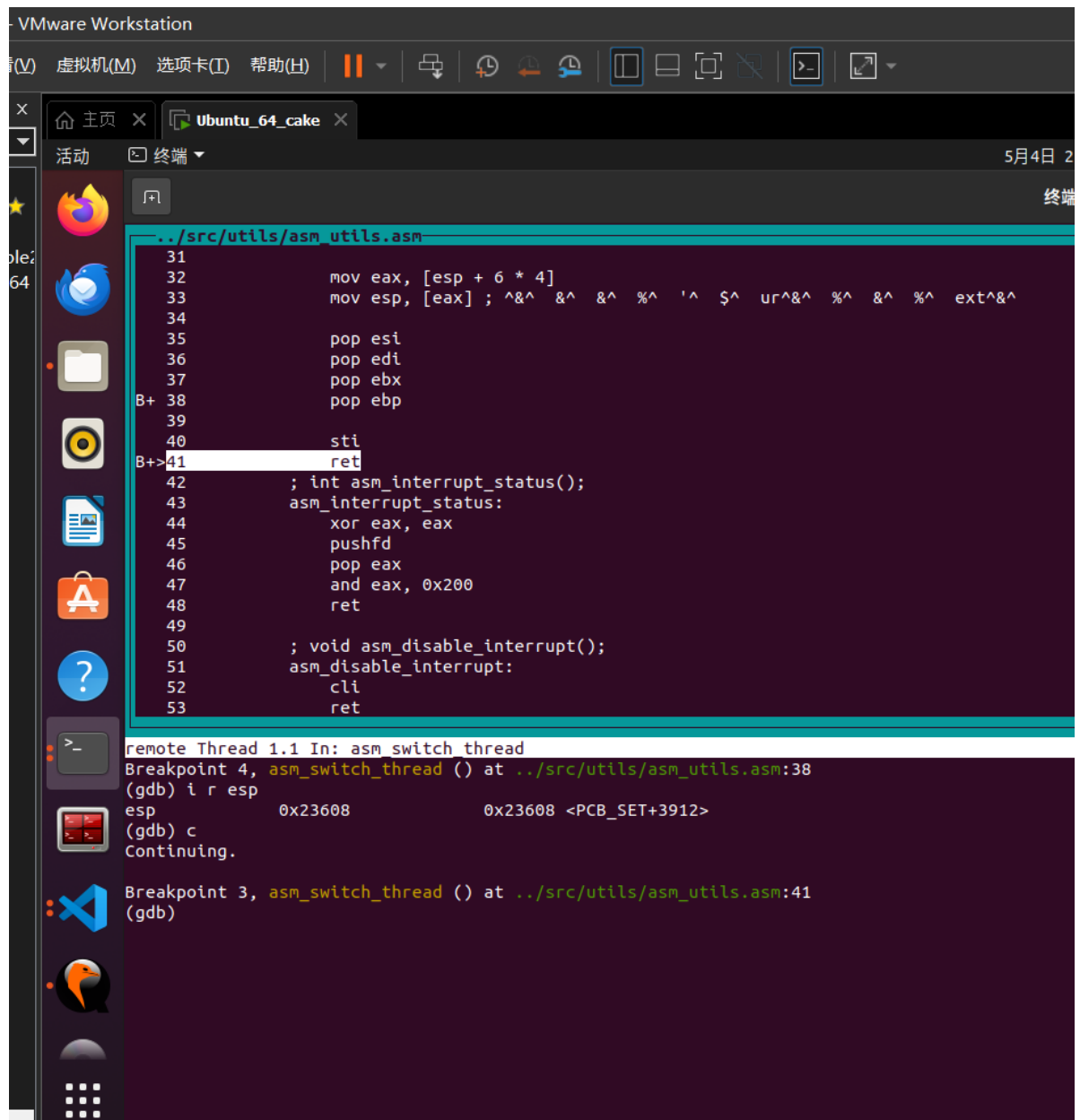


```
主页 x Ubuntu_64_cake x 5月
终端
../src/kernel/setup.cpp
44     }
45     };
46     void my_thread_schedule0(void* arg) {
47         // int k = 3;
48         // while(k--) {
49             //     printf("the running thread is 0\n");
50             // }
51         // return ;
52         long long count = 0;
53         while(true) {
54             count ++;
55             if(count == 10000000) {
56                 printf("the running thread is 0\n");
57                 count = 0;
58             }
59         };
60     }
61     void child1(void* arg) {
62         int k = 3;
63         int count = 0;
64         while(true) {
65             count ++;
66             if(count == 100000000) {

remote Thread 1.1 In: my_thread_schedule0
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
0x00020f2f in c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:100
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
asm_time_interrupt_handler () at ../src/utls/asm_utls.asm:69
(gdb) ni
asm_time_interrupt_handler () at ../src/utls/asm_utls.asm:70
(gdb) ni
my_thread_schedule0 (arg=0x0) at ../src/kernel/setup.cpp:54
(gdb) i r eip
eip          0x207fe          0x207fe <my_thread_schedule0(void*)+26>
(gdb)
```

可以看到我们依据返回地址返回到调度函数，再从调度函数返回到时钟处理函数，再从时钟处理函数返回到了线程函数中（这里也是在栈上新建了这些函数的调用栈，然后依次返回的），这样就可以往下继续执行了，这样我们就完成了依据返回地址返回到中断点继续执行。

下面的几张图展示了从调度函数到原线程函数过程中esp的变化情况，可以看到栈顶不断向高地址缩，直到回到原线程函数的栈：



```
机(M) 选项卡(T) 帮助(H) 终端 5月4日 2
Ubuntu_64_cake 终端
Thunderbird 邮件/新闻

../src/kernel/program.cpp
98         releasePCB(running);
101     }
102     ListItem *item = readyPrograms.front();
103     PCB *next = ListItem2PCB(item, tagInGeneralList);
104     PCB *cur = running;
105     next->status = ProgramStatus::RUNNING;
106     running = next;
107     readyPrograms.pop_front();
108     asm_switch_thread(cur, next);
109     interruptManager.setInterruptStatus(status);
110
111 }
112
113 void ProgramManager::FCFS() {
114     bool status = interruptManager.getInterruptStatus();
115     interruptManager.disableInterrupt();
116     releasePCB(running);
117     if (readyPrograms.size() == 0)
118     {
119         interruptManager.setInterruptStatus(status);
120         //'^^ %^ ^ '^ %^ )^ %^ $^ $^ '^
}

remote Thread 1.1 In: ProgramManager::schedule
(gdb) x/4xw 0x23610
0x23610 <PCB_SET+3920>: 0x000226c0      0x000236c0      0x00000020      0x00000018
(gdb) x/i 0x226c0
0x226c0 <PCB_SET>:    cld
(gdb) x/i 0x236c0
0x236c0 <PCB_SET+4096>:    cld
(gdb) ni
(gdb) ni
(gdb) i r esp
esp      0x23620      0x23620 <PCB_SET+3936>
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) i r esp
esp      0x23620      0x23620 <PCB_SET+3936>
(gdb)
```

VMware Workstation

(V) 虚拟机(M) 选项卡(T) 帮助(H)

Ubuntu_64_cake

活动 终端 5月4日

```
../src/kernel/interrupt.cpp
90 // ^%^ %^ ' ^ CFS^'^ &^ &^ %^ &^ )^ $^ &^ &^ %^ ( ^ ^
91 PCB *cur = programManager.running;
92
93 if (cur->ticks)
94 {
95     --cur->ticks;
96     ++cur->ticksPassedBy;
97 }
98 else
99 {
100     programManager.schedule();
101 }
102
103 // //^$^ )^ &^ SA^&^ %^ %^ ' ^ &^ ' ^ &^ )^ $^ &^ / ^ ( ^ ^
104 // PCB* cur = programManager.running;
105 // if(cur -> ticks) {
106 //     --cur -> ticks;
107 //     ++cur -> ticksPassedBy;
108 // } else {
109 //     programManager.PSA();
110 // }
111 }
```

remote Thread 1.1 In: c_time interrupt handler

```
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) i r esp
esp          0x23620          0x23620 <PCB_SET+3936>
(gdb) x/8xw 0x23620
0x23620 <PCB_SET+3936>: 0x000226c0    0x000236c0    0x000236ec    0x00000000
0x23630 <PCB_SET+3952>: 0x00000000    0x00000000    0x00023668    0x00020f2f
(gdb) ni
(gdb) ni
0x00020f2f in c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:100
(gdb) i r esp
esp          0x23640          0x23640 <PCB_SET+3968>
(gdb) i r eip
eip          0x20f2f          0x20f2f <c_time_interrupt_handler()+73>
(gdb) |
```




```
../src/utils/asm_utils.asm
```

```
59     asm_time_interrupt_handler:
60         pushad
61
62         ; ^%^ )^ ^ EOI^&^ &^ ^ / ^ %^ %^ $^ $^ ^&^ $^ &
63         mov al, 0x20
64         out 0x20, al
65         out 0xa0, al
66
67         call c_time_interrupt_handler
68
69     popad
70     iret
71
72     ; void asm_in_port(uint16 port, uint8 *value)
73     asm_in_port:
74         push ebp
75         mov ebp, esp
76
77         push edx
78         push eax
79         push ebx
80
81         xor eax, eax
```

remote Thread 1.1 In: asm_time_interrupt_handler

0x23630 <PCB_SET+3952>: 0x00000000 0x00000000 0x00023668 0x00020f2f

(gdb) ni

(gdb) ni

0x00020f2f in c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:100

(gdb) i r esp

esp 0x23640 0x23640 <PCB_SET+3968>

(gdb) i r eip

eip 0x20f2f 0x20f2f <c_time_interrupt_handler()+73>

(gdb) ni

(gdb) ni

(gdb) ni

(gdb) ni

asm_time_interrupt_handler () at ../src/utils/asm_utils.asm:69

(gdb) i r es

esp 0x23670 0x23670 <PCB_SET+4016>

(gdb) i r eip

eip 0x21b7c 0x21b7c <asm_time_interrupt_handler+12>

(gdb) █

```
../src/kernel/setup.cpp
44     }
45     };
46     void my_thread_schedule0(void* arg) {
47         // int k = 3;
48         // while(k--) {
49             //     printf("the running thread is 0\n");
50         // }
51         // return ;
52         long long count = 0;
53         while(true) {
54             count ++;
55             if(count == 10000000) {
56                 printf("the running thread is 0\n");
57                 count = 0;
58             }
59         };
60     }
61     void child1(void* arg) {
62         int k = 3;
63         int count = 0;
64         while(true) {
65             count ++;
66             if(count == 100000000) {

remote Thread 1.1 In: my_thread_schedule0
(gdb) i r eip
eip          0x20f2f          0x20f2f <c_time_interrupt_handler()+73>
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
asm_time_interrupt_handler () at ../src/utils/asm_utils.asm:69
(gdb) i r esp
esp          0x23670          0x23670 <PCB_SET+4016>
(gdb) i r eip
eip          0x21b7c          0x21b7c <asm_time_interrupt_handler+12>
(gdb) ni
asm_time_interrupt_handler () at ../src/utils/asm_utils.asm:70
(gdb) ni
my_thread_schedule0 (arg=0x0) at ../src/kernel/setup.cpp:54
(gdb) i r esp
esp          0x2369c          0x2369c <PCB_SET+4060>
(gdb) |
```

经过上面的追踪，详细的说明了一个新创建的线程是如何被调度然后开始执行的。一个正在执行的线程是如何被中断然后被换下处理器的，以及换上处理机后又是如何从被中断点开始执行的所有过程。

任务四：调度算法的实现

在这一部分实现了两个调度算法：FCFS算法和优先级调度算法。

FCFS算法

FCFS算法的实现比较简单，即按照就绪队列中的顺序依次运行线程即可。首先明确**调度只发生在线程运行完成时**，所以调用调度函数只在program_exit()函数中，并且不需要时钟中断的参与，不需要时间片和优先级。在调度算法中，**首先关中断，释放正在运行的线程的PCB，判断就绪队列是否为空，若为空则等待，直到队列不为空。取出就绪队列第一个listitem，找到它的PCB，调整就绪队列，通过asm_switch_thread将其切换到cpu上运行，开中断。**

```

void ProgramManager::FCFS() {
    bool status = interruptManager.getInterruptStatus();
    interruptManager.disableInterrupt();
    releasePCB(running);
    if (readyPrograms.size() == 0)
    {
        interruptManager.setInterruptStatus(status);
        //等待直到队列不为空
        while(readyPrograms.size() == 0);
    }
    ListItem *item = readyPrograms.front();
    PCB *next = ListItem2PCB(item, tagInGeneralList);
    next->status = ProgramStatus::RUNNING;
    running = next;
    readyPrograms.pop_front();
    asm_switch_thread(0, next);
    interruptManager.setInterruptStatus(status);
}

```

因为不是抢占式算法，所以pid为0的线程也必须要被调度，不然其他程序无法运行。

测试程序中，首先初始创建三个子线程，编号分别是0、1、2，在1、2线程运行时创建子线程child1、child2，每个线程都是打印三遍信息，来证明线程没有被中断，按照理论预测，进入就绪队列的顺序依次是0、1、2、child1、child2。执行顺序也是如此，运行程序，实际运行情况与预期一致，证明算法的正确性：

```

Machine View
SeaBIOS (version 1.13.0-1ubuntu1.1)

iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB00 CA00

Booting from Hard Disk...
the running thread is 0
the running thread is 0
the running thread is 0
the running thread is 1
the running thread is 1
the running thread is 1
the running thread is 2
the running thread is 2
the running thread is 2
the running thread is the child for thread1
the running thread is the child for thread1
the running thread is the child for thread1
the running thread is the child for thread2
the running thread is the child for thread2
the running thread is the child for thread2
int pid = programManager.executeThread(child

```

优先级调度算法

优先级调度算法依据线程的优先级进行调度。通过优先级来确定调度时谁被换上cpu。这里规定优先级整数越小优先级越高，时间片与优先级成正比。调度发生在**当前线程函数执行完成、时间片完全消耗掉、新的进程到来时**。属于抢占式算法。首先要在时钟中断处理函数、program_exit()、executeThread () 中添加我们的调度算法函数：

```

// 中断处理函数
extern "C" void c_time_interrupt_handler()
{
    // 在实现FCFS算法时和时钟中断没有关联，所以要注释掉这一部分的代码
    // PCB *cur = programManager.running;

    // if (cur->ticks)
    // {
    //     --cur->ticks;
    //     ++cur->ticksPassedBy;
    // }
    // else
    // {
    //     programManager.schedule();
    // }

    //下面是PSA抢占式算法的时钟中断，这里给定的预估时间复用了RR算法中的时间片，并且分配也是按照优先级来分配的
    PCB* cur = programManager.running;
    if(cur -> ticks) {
        --cur -> ticks;
        ++cur -> ticksPassedBy;
    } else {
        programManager.PSA();
    }
}

```

```

void program_exit()
{
    PCB *thread = programManager.running;
    thread->status = ProgramStatus::DEAD;
    //开启这个要注释掉下面的内容
    // programManager.FCFS();

    if (thread->pid)
    {
        // programManager.schedule();
        programManager.PSA();
    }
    else
    {
        interruptManager.disableInterrupt();
        printf("halt\n");
        asm_halt();
    }
}

```

```

thread->status = ProgramStatus::READY;
thread->priority = priority;
thread->ticks = priority * 10;
thread->ticksPassedBy = 0;
thread->pid = ((int)thread - (int)PCB_SET) / PCB_SIZE;

// 线程栈
thread->stack = (int *)((int)thread + PCB_SIZE);
thread->stack -= 7;
thread->stack[0] = 0;
thread->stack[1] = 0;
thread->stack[2] = 0;
thread->stack[3] = 0;
thread->stack[4] = (int)function;
thread->stack[5] = (int)program_exit;
thread->stack[6] = (int)parameter;

allPrograms.push_back(&(thread->tagInAllList));
readyPrograms.push_back(&(thread->tagInGeneralList));

// 恢复中断
interruptManager.setInterruptStatus(status);

printf("call for create thread!\n");
ProgramManager::PSA();

return thread->pid;
}

```

接下来进行函数的实现：

```

void ProgramManager::PSA()
{
    bool status = interruptManager.getInterruptStatus();
    interruptManager.disableInterrupt();

    if (readyPrograms.size() == 0)
    {
        interruptManager.setInterruptStatus(status);
        return;
    }

    // int min = running -> priority;
    PCB* next = nullptr;
    ListItem* minlist = nullptr;
    if(running->status != ProgramStatus::DEAD) {
        next = running;
        minlist = &(running -> tagInGeneralList);
    } else {
        minlist = readyPrograms.front();
        next = ListItem2PCB(minlist, tagInGeneralList);
    }
    ListItem *item = readyPrograms.front();
    while(item) {
        PCB* it = ListItem2PCB(item, tagInGeneralList);
        if(it -> priority < next -> priority || ((next == running) && (it -> priority == next -> priority))) {
            next = it;
            // min = it -> priority;
        }
        item = readyPrograms.front();
    }
}

```

```

        minlist = item;
    }
    item = item -> next;
}

// PCB *next = ListItem2PCB(item, tagInGeneralList);

PCB *cur = running;
if (cur -> status == ProgramStatus::RUNNING)
{
    cur->status = ProgramStatus::READY;
    readyPrograms.push_back(&(cur->tagInGeneralList));
    if (cur -> ticks == 0)
    {
        cur->ticks = cur->priority * 10;
    } else if (running->status == ProgramStatus::DEAD) {
        releasePCB(cur);
    }

    next->status = ProgramStatus::RUNNING;
    running = next;
    readyPrograms.erase(minlist);

    asm_switch_thread(cur, next);

    interruptManager.setInterruptStatus(status);
}

```

首先关中断，判断队列，为空返回。接着找到队列（包括running，若是running为dead，则一定被换下，若与running优先级相同，也会把running换下，防止优先级相同的情况下一直占据cpu）中优先级最高的线程，依据线程状态处理running线程，然后换上下一个线程。注意，在running为nullptr时，我们不会换上任何线程，这意味着直到setup_kernel函数主动将第一个线程搬上cpu之前，executeThread () 处的PSA () 永久失效。这是因为running为nullptr时，priority为固定值-268370093，显然比所有priority都要小，所以cpu换上的是nullptr，直到setup_kernel主动将第一个线程放到cpu上，running不为nullptr，此时executeThread () 处的PSA就奏效了。

这是符合预期的，因为第一个进程不会被终止，所以假如创建它时就换上来，此时队列里面就它一个，它又不创建子进程，那么它就会一直运行下去，阻碍剩下的进程的创建，所以我们要求在创建完所有的父进程之后，再将第一个进程显示放在cpu上，这样可以保证调度。

如果想要一开始就让executeThread () 处的PSA奏效，即第一个进程里面创建了之后所有的子进程，也很简单，只要在下图所示的地方按注释修改就可以：

```

146     PCB* next = nullptr;
147     ListItem* minlist = nullptr;
148     //这里想要让running初始为nullptr时也可以切换，只需要在判断时改成如下判断：
149     // if(running && running->status != ProgramStatus::DEAD)
150     if(running->status != ProgramStatus::DEAD) {
151         next = running;
152         minlist = &(running -> tagInGeneralList);
153     } else {
154         minlist = readyPrograms.front();
155         next = ListItem2PCB(minlist, tagInGeneralList);
156     }

```

测试代码下，将第一个线程（线程0）变成死循环线程，线程1（非死循环）创建一个死循环线程child1，线程2（非死循环）创建死循环线程child2，它们的优先级如下：

name	priority
0	10
1	5
2	3
child1	6
child2	6

```
// 进程/线程管理器
programManager.initialize();
param p(2,2);
// 创建第一个线程
int pid = programManager.executeThread(my_thread_schedule0, nullptr, "my thread schedule0", 10);
if (pid == -1)
{
    printf("can not execute thread\n");
    asm_halt();
}
int pid1 = programManager.executeThread(my_thread_schedule1, nullptr, "my thread schedule1", 5);
if (pid1 == -1)
{
    printf("can not execute thread\n");
    asm_halt();
}
int pid2 = programManager.executeThread(my_thread_schedule2, nullptr, "my thread schedule2", 3);
if (pid2 == -1)
{
    printf("can not execute thread\n");
    asm_halt();
}
ListItem *item = programManager.readyPrograms.front();
PCB *firstThread = ListItem2PCB(item, tagInGeneralList);
firstThread->status = RUNNING;
programManager.readyPrograms.pop_front();
programManager.running = firstThread;
asm_switch_thread(0, firstThread);

asm_halt();
```

```

void my_thread_schedule0(void* arg) {
    // int k = 3;
    // while(k--) {
    //     printf("the running thread is 0\n");
    // }
    // return ;
    long long count = 0;
    while(true) {
        count ++;
        if(count == 100000000) {
            printf("the running thread is 0\n");
            count = 0;
        }
    };
}

void child1(void* arg) {
    int k = 3;
    int count = 0;
    while(true) {
        count ++;
        if(count == 100000000) {
            printf("    the running thread is the child for thread1\n");
            count = 0;
        }
    }
}
}

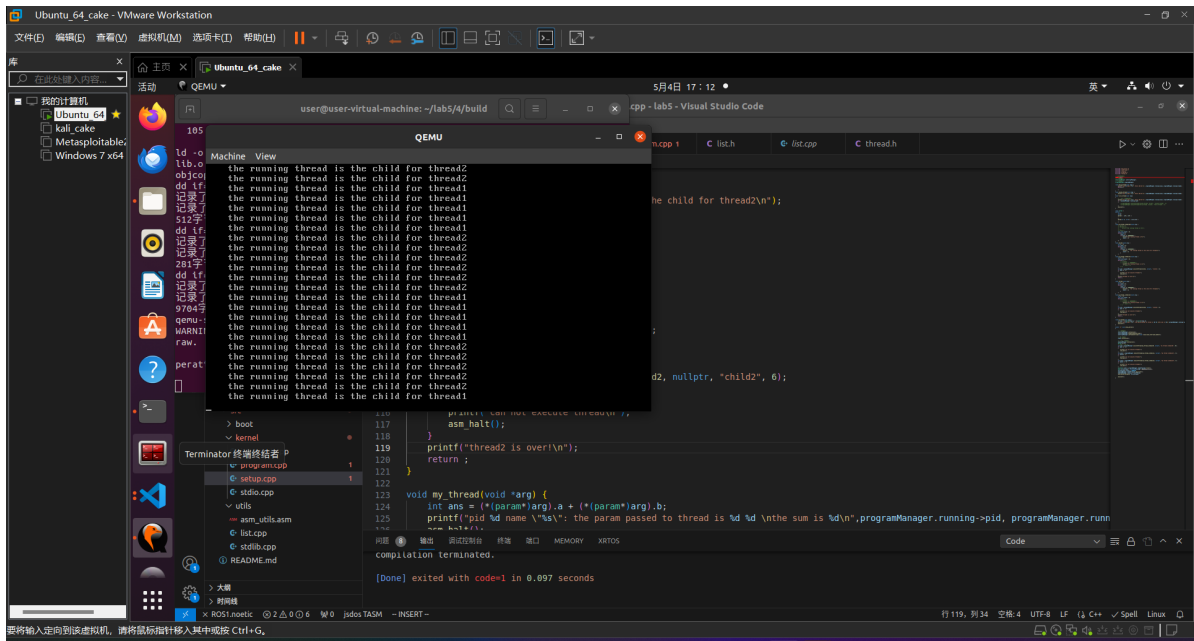
```

```

> kernel > C:\setup.cpp > setup_kernel()
void my_thread_schedule1(void* arg) {
    int k = 3;
    long long count = 0;
    while(k--) {
        // ++count;
        // if(count == 100000000) {
        //     printf("the running thread is 1\n");
        //     // count = 0;
        // }
    }
    int pid = programManager.executeThread(child1, nullptr, "child1", 6);
    if (pid == -1)
    {
        printf("can not execute thread\n");
        asm_halt();
    }
    printf("thread1 is over!\n");
    return ;
}

void child2(void* arg) {
    int k = 3;
    int count = 0;
    while(true) {
        count ++;
        if(count == 100000000) {
            printf("    the running thread is the child for thread2\n");
            count = 0;
        }
    }
}
}

```

总结

通过这次实验，我学到了线程实现、创建与调度的相关知识，并且也学习到了可变参函数与宏的有关知识，受益匪浅。